

Reviewer Pack — Minimal Order of Quandle Representations and Communication C...

Entry f874ffbd83fa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 11:11:04 UTC

Minimal Order of Quandle Representations and Communication Complexity Rank Correlation

Entry ID: f874ffbd83fa **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 11:11:04 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Communication Complexity

Statement:

For every n -ary communication protocol φ , the minimal order of a quandle representation that can simulate φ is $\Theta(n^{3/2})$.

Rationale (proposer's reasoning):

Quandles are algebraic structures with non-trivial symmetry properties. Their representations may capture essential aspects of communication protocols that involve symmetric operations. This conjecture aims to explore the potential of quandle theory in characterizing the complexity of communication tasks.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 78f0c05ded422257

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the order of the quandle representation and the communication complexity rank is greater than or equal to 0.7, calculated from 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quandle theory" AND "communication complexity" AND "minimal order" - "quandle representation" simulating "communication protocol" with threshold $n^{(3/2)}$ " - "simulating communication protocols" using "quandle structures"

Top relevant hits considered: - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R - [http://arxiv.org/abs/2204.12571v1] Multiplication of quandle structures - [http://arxiv.org/abs/2112.15454v4] Advanced Drone Swarm Security by Using Blockchain Governance Game - [http://arxiv.org/abs/1009.4674v3] Intuitive representation of surface properties of biomolecules using BioBlender

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_protocol(n):
        # Generate a simple n-ary communication protocol
        return [random.randint(1, 2**n - 1) for _ in range(n)]

    def construct_quandle_representation(protocol):
        # Construct a minimal quandle representation (simplified example)
        quandle = {}
        for x in protocol:
            quandle[x] = {y: (x + y) % len(protocol) for y in protocol}
        return quandle

    def order_of_quandle(quandle):
        # Calculate the order of the quandle
        return max(len(quandle[x]) for x in quandle)

    def communication_complexity_rank(protocol):
        # Simplified example: rank is the number of unique elements
        return len(set(protocol))

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
```

```

for n in n_values:
    protocol = generate_protocol(n)
    quandle = construct_quandle_representation(protocol)
    order = order_of_quandle(quandle)
    rank = communication_complexity_rank(protocol)
    results.append((n, order, rank))

if not results:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n_values, orders, ranks = zip(*results)
mean_order = sum(orders) / len(orders)
mean_rank = sum(ranks) / len(ranks)
correlation = (sum((x - mean_order) * (y - mean_rank) for x, y in zip(orders, ranks)) /
               math.sqrt(sum((x - mean_order)**2 for x in orders) *
                           sum((y - mean_rank)**2 for y in ranks)))

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67,
        71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        mean_value = None
        std_value = None
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

```

```
print(f"RESULT: {'SUPPORTED' if all(result['conjecture_holds'] for result in results) else 'FA
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
'instances_tested': 6, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
TRIAL: {'metric_name': 'correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conje
RESULT: SUPPORTED mean=1.0 std=0.0 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only considers a very small range of n (up to 40), which is insufficient to validate the conjecture that the minimal order of quandle representations scales with $n^{(3/2)}$. The metric may not scale trivially with n , and the results could be coincidental for such a small sample size.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code only considers a very small range of n (up to 40), which is insufficient to validate the conjecture that the minimal order of quandle representations scales with $n^{(3/2)}$. The critic challenged the results due to the limited sample size, and the pre-registered support condition was not unambiguously met. | next: Expand the range of n values tested to a larger scale to validate the conjecture. Additionally, consider using more robust statistical methods to ensure the reliability of t

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19388
2	preregistration	ollama_remote	glm4:latest	0	0	12263
3	novelty	ollama_remote	glm4:latest	0	0	8393
4	novelty	ollama_remote	glm4:latest	0	0	9983

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21854
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9569
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14976
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10306
9	critic	ollama_remote	glm4:latest	0	0	11318
10	judge	ollama_remote	glm4:latest	0	0	11053

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129103 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/f874ffbd83fa.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f874ffbd83fa.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f874ffbd83fa.tar.gz* (if generated)